Forgettable Athlete

A Simple, Story-wise Explanation of the Whole Project
Privacy-Preserving Injury Risk Prediction with Machine Learning,
Explainable AI (SHAP) and Machine Unlearning (SISA)

Course: Data Science and AI Project (Spring) — Tokyo International University

Team: Asir Abdullah (24224511), Talha MD Abu (24224559), Nazmul Islam

What is this document?

Think of this PDF as your friendly tour guide. The Jupyter notebook *forgettable_athlete.ipynb* and the PowerPoint deck *Forgettable_Athlete_Presentation.pptx* together hold the whole story of our project. This PDF re-tells that story in plain language. No math walls, no jargon without explanation. By the end you will understand:

- **Why** we built this project (the real-world problem).
- **What** data we used and how we cleaned it.
- **How** we engineered new features and why each one matters.
- **Which** machine learning models we tried and how each one works.
- **Why** we got such high scores and what each score means.
- **How** SHAP explains every prediction.
- **How** machine unlearning (SISA) lets us forget a player without retraining everything.
- **What** libraries we used and what each library is doing for us.

Chapter 1 — The Story: Why This Project Exists

Imagine a football club. Every week the medical and fitness staff collect data on their players: training hours, sleep, stress, sprint speed, how many matches they played, whether they had injuries last season, and so on. Clubs feed all this data into computer models so they can answer one question that keeps the manager awake at night:

> *"Is this player likely to get injured next season?"*

If the answer is **yes**, the medical team can step in early — give the player extra rest, change the training plan, fix sleep habits, work on weak muscles. That single decision can save a career.

But then a second problem appears. Players are people, not statistics. They transfer to other clubs. They retire. They invoke privacy laws (like Europe's GDPR "right to be forgotten") and say:

> *"Please delete all my data and remove my influence from your models."*

Here is the hard part. A trained model is like a cake that already has the player's ingredients baked in. You can't just scoop them out. The textbook fix is to retrain the model from scratch on the remaining data — like baking a brand new cake every time someone wants their flour back. Modern boosting models on big datasets take hours to retrain. Do that every time a player leaves and your servers cry.

So our project has two missions, hand-in-hand:

- **Mission 1 — Predict injuries** as accurately as possible using machine learning.
- **Mission 2 — Respect privacy** by being able to *forget* a player quickly, without losing the accuracy we worked so hard to get.

That is why we named the project **"Forgettable Athlete"**. It is a system that predicts injuries today and can cleanly forget a player tomorrow.

Chapter 2 — The Dataset: Where the Story Starts

We used a public Kaggle dataset called the **University Football Injury Prediction Dataset** (*yuanchunhong/university-football-injury-prediction-dataset*). It contains records of 800 university-level football players with information about their bodies, training, lifestyle, physical tests and whether they actually got injured the next season.

On top of that real data we added three small **synthetic** columns that the original dataset did not have: *Player_ID*, *Team_ID* and *Deletion_Request*. We made them up ourselves because the original CSV did not include any privacy or identity information, and our project needs to simulate "Player X wants to be forgotten". The injury labels themselves are **not** synthetic — only the IDs are.

The numbers at a glance

Players (rows)	800
Total columns	25
Injury_Next_Season = 1 (injured)	400 (50%)
Injury_Next_Season = 0 (no injury)	400 (50%)
Playing positions	Defender, Forward, Goalkeeper, Midfielder
Train / Test split	640 / 160 (80% / 20%)

Why a 50/50 balance matters: in many medical datasets, only a small fraction of people get injured, so a lazy model that always predicts "no injury" already looks 90% accurate. Our dataset is balanced, so a high accuracy score actually means the model learned something real. We did not need fancy resampling tricks like SMOTE.

The five families of features

We grouped the 25 columns into five intuitive families. Think of them as five different lenses through which we look at each player:

Family	What it tells us	Example columns
Demographics	Who the player is, body shape	Age, Height, Weight, BMI, Position
Workload	How much they're being pushed	Training hours/week, Matches played, Previous injuries
Physical tests	Strength, flexibility, speed, agility	Knee strength, Hamstring flex, Reaction time, Balance, Sprint,
Lifestyle	How well they recover and live	Sleep hours, Stress level, Nutrition, Warmup adherence
Privacy (synthetic)	Identity & deletion (NOT features for the model)	Player_ID, Team_ID, Deletion_Request

The target column we want to predict: *Injury_Next_Season* — a simple yes/no flag (1 = injured, 0 = healthy).

Chapter 3 — Cleaning the Data (Preprocessing)

Raw data is messy. Before any model can learn from it, we tidy it up. Think of preprocessing as washing the vegetables before cooking. We did five small steps:

- **Step 1 — Missing values.** We checked every column. Lucky us: there were no gaps. If there had been, we would have filled numeric gaps with the median (the middle value), because the median is robust to outliers.
- **Step 2 — Duplicates.** We checked if the same player appeared twice. No duplicates were found.
- **Step 3 — One-hot encoding the Position column.** A model cannot read the word “Forward”. So we turned *Position* into four 0/1 columns: *Pos_Defender*, *Pos_Forward*, *Pos_Goalkeeper*, *Pos_Midfielder*. Each player is now 1 in exactly one of those columns.
- **Step 4 — Keep ID columns aside.** Player_ID, Team_ID and Deletion_Request are NOT predictors — knowing a Player_ID tells the model nothing about injuries. We hold them in a separate table so we can use them later when someone requests deletion.
- **Step 5 — Final matrix.** After cleaning, we have 800 rows and 28 columns ready for feature engineering.

Chapter 4 — Feature Engineering: Inventing Smarter Columns

This is the chapter where we go from *data scientist* to *sports scientist*. Raw features like “training hours” or “previous injuries” are useful, but they don’t directly tell the model **how stressed** a body is, or **how well** it has recovered. So we combined raw numbers into four hand-crafted features that sports science literature already knows are linked to injury risk.

Why bother? Because feature engineering is often **the single biggest lever** you have to boost a model’s score. Models are dumb on their own; they just see numbers. If you give them a column that already captures the right idea, learning becomes easier.

1. ACWR — Acute-Chronic Workload Ratio

This is a famous sports science number. It compares this week’s training load to your recent average load. If you trained 12 hours this week and your rolling average over recent weeks was 8, the ratio is 1.5. Studies show that **a ratio greater than ~1.3 is a known injury risk zone** — the body did not have time to adapt.

> Formula in plain English: *this week’s training hours ÷ rolling mean of last 5 weeks’ training hours*.

2. Recovery Score

A weighted blend of the things that help a body recover:

- 35% Sleep (more sleep = better recovery)
- 30% Nutrition quality (better food = better recovery)
- 20% Warmup routine adherence (warmups protect muscles)
- 15% Inverse of stress (less stress = better recovery)

Result: one number between 0 and 1. Higher = better recovered.

3. Fatigue Index

The opposite side of the coin. We combine training load and matches played, then subtract recovery. High fatigue = high injury risk.

> Formula in plain English: 50% training load + 30% matches played + 20% (1 - recovery).

4. Injury Frequency Score

Previous injuries divided by age. A 20-year-old with 4 past injuries is a much bigger concern than a 24-year-old with 4 past injuries. This proxies **chronic injury proneness**.

Why these features helped us get a better score

- **They encode interactions** the model would otherwise have to discover from scratch. Telling the model directly “this person is fatigued” is faster than hoping it figures it out from training hours alone.

- **They lift weak raw signals.** Sleep on its own correlates only weakly with injury. But Recovery_Score, which combines sleep with nutrition, warmup and stress, correlates more strongly.
- **They are explainable.** When SHAP later says “Fatigue_Index pushed this player toward injury”, a coach immediately knows what to do — reduce training load or improve recovery.
- **They beat the curse of small data.** With only 800 rows, a model cannot learn very complex patterns from raw features. Engineered features give it a head-start.

Chapter 5 — Looking at the Data Before Modeling (EDA)

Before any model is trained, a good data scientist **looks** at the data. We made six plots and a correlation heatmap. Here is what we learned:

- **Class balance:** 400 injured vs 400 healthy. Perfect 50/50.
- **Training hours vs injury:** the injured group trains slightly more on average. Suggests overtraining is a real factor.
- **Age:** distributions overlap completely between injured and healthy. Age alone is a **weak** predictor.
- **Sleep:** injured players sleep a little less. Small but real signal.
- **Previous injuries:** the violin plot shows clearly that players with more past injuries are more likely to get injured again. **Strong** signal.
- **BMI:** distributions overlap. BMI by itself is almost useless here.

Correlations: the heatmap story

None of the single columns dominate. The strongest correlations with *Injury_Next_Season* are still modest. This is normal — injuries are **multi-causal**; they come from a combination of factors.

Three big takeaways:

- Because no single column is a silver bullet, we need models that can mix many weak signals together → tree ensembles like Random Forest, XGBoost, LightGBM.
- Our engineered *Fatigue_Index* and *Injury_Frequency_Score* show up with stronger correlation than most raw columns — feature engineering paid off.
- Physical-test columns are correlated with each other. Tree models will quietly pick the most informative one and ignore the duplicates.

Chapter 6 — Train / Test Split and Scaling

Now we split the data into two piles:

- **Training set:** 640 players (80%) — the model studies these.
- **Test set:** 160 players (20%) — these are kept secret from the model and used at the end to measure performance honestly.

We used **stratified** splitting, which means the 50/50 injury balance is preserved in both piles. Without stratification you could randomly end up with, say, 80% injured players in the test set, which would skew results.

Then we applied **StandardScaler**. This rescales every numeric column so it has mean 0 and standard deviation 1. Why? Some models (especially Logistic Regression) are very sensitive to feature scales. Without scaling, a column measured in kilograms would unfairly dominate a column measured between 0 and 1. Important detail: we **fit** the scaler only on training data, then **transform** both train and test — otherwise information from the test set leaks into the model.

Chapter 7 — The Four Models and How They Work

We trained four different machine learning algorithms. We did this on purpose: each algorithm has a different way of seeing the world, and comparing them tells us which approach fits this problem best.

Model 1 — Logistic Regression (the linear baseline)

Imagine you are drawing a straight line on a 2D map to separate red dots from blue dots. Logistic Regression does that, but in 25 dimensions instead of two. It assigns each feature a coefficient (a weight). Positive weight = feature pushes toward injury; negative weight = feature pushes toward healthy. It outputs a probability between 0 and 1 using the sigmoid function.

Why try it? Because it is fast, interpretable (you can literally read the weights), and a great baseline. If a fancy model can't beat logistic regression by much, the dataset is probably linearly separable.

Weakness: it cannot capture interactions on its own. If injury risk depends on "high training AND low sleep *at the same time*", logistic regression cannot see that combination.

Model 2 — Random Forest (the wisdom of the crowd)

Imagine asking 300 different doctors "will this player get injured?" Each doctor sees only a random subset of the data and a random subset of the features. They each grow a decision tree. Then you take the majority vote. That is a Random Forest — 300 trees voting together.

Why it works: any single tree might overfit (memorise the training data), but 300 different trees, each seeing a different angle, average out their mistakes. Random Forest handles non-linear feature interactions for free and does not need any scaling.

Weakness: the final model is huge and not as easy to interpret as a single decision tree or a logistic regression.

Model 3 — XGBoost (gradient boosting)

Boosting is a clever idea: instead of building 300 independent trees, build them **one after the other**, where each new tree focuses on the mistakes of the previous trees. The first tree makes some predictions. The second tree tries to fix what the first one got wrong. The third fixes what the first two got wrong. And so on.

XGBoost (Extreme Gradient Boosting) is a famous implementation of this idea, tuned to be fast and very accurate on tabular data — the kind of data we have here. It often wins Kaggle competitions on structured data.

Why try it? Because gradient boosting tends to be the gold standard for tabular ML.

Weakness: hyperparameter sensitive — you can over- or under-fit easily if you don't tune carefully.

Model 4 — LightGBM (lightning-fast boosting)

LightGBM (Light Gradient Boosting Machine) is Microsoft's version of gradient boosting. The math is similar to XGBoost but it builds trees in a **leaf-wise** way (it grows the leaf with the biggest improvement first), which makes it dramatically faster on large datasets and lower on memory.

Why we like it here: we needed it specifically for the **machine unlearning** module — when a deletion request comes in, we have to retrain shards quickly. LightGBM is fast enough to do that almost instantly. Also, it handles categorical features natively.

Weakness: can overfit on very small data without regularisation.

Why use four models instead of one?

- **Coverage:** linear (Logistic) + bagging (Random Forest) + boosting (XGBoost, LightGBM). We sample three different families of ML algorithms.
- **Fair comparison:** every model is trained on the same split, the same scaled features, with default hyperparameters. The winner wins on merit.
- **Insurance:** if one model has a bad day, the others act as a sanity check.

Chapter 8 — The Results (and Why the Scores Are So High)

Here is the scoreboard from the notebook:

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Time (s)
Logistic Regression	0.950	0.950	0.950	0.950	0.996	0.011
Random Forest	0.956	0.962	0.950	0.956	0.993	0.348
LightGBM	0.919	0.947	0.888	0.916	0.988	2.156
XGBoost	0.938	0.961	0.913	0.936	0.985	0.129

What each metric means (in plain language)

- **Accuracy** — out of 160 test players, how many did we classify correctly? 0.95 means 152 out of 160 were correct.
- **Precision** — when the model says “injury”, how often is it right? High precision means few *false alarms*.
- **Recall** — out of all the players who actually got injured, how many did we catch? High recall means few *missed injuries*.
- **F1** — the harmonic mean of precision and recall. A balanced single number.
- **ROC-AUC** — across every possible threshold, how well does the model rank injured players above healthy ones? 1.0 is perfect, 0.5 is random guessing. Our 0.99 means the model almost always ranks an injured player above a healthy one.

Why are the scores so high?

- **The dataset is balanced and well-engineered.** 50/50 classes + our four engineered features mean a strong signal is already present.
- **The features carry a lot of information.** Previous injury count, fatigue index, and recovery score together are very strong predictors.
- **The dataset is relatively small (800 rows)** and the features are well-curated, so models converge to a clean decision boundary.
- **Honest test set.** We evaluated on 160 rows the model never saw. The 0.99 AUC is not a leakage artefact — we kept the scaler fit only on training data.

Which model wins?

By ROC-AUC, **Logistic Regression** is on top (0.996), with Random Forest a whisker behind (0.993). By accuracy, **Random Forest** edges out at 0.956. The boosting models (XGBoost, LightGBM) are slightly behind here because the dataset is small and the relationships happen to be close to linear — boosting shines more on larger, messier datasets.

So why do we still use LightGBM downstream? Because for SHAP explanations and SISA unlearning we need a fast tree-based model — LightGBM is the right tool there, not because it has the best raw AUC, but because its tree structure exposes the explanations and supports quick shard retraining.

Chapter 9 — SHAP: Making the Model Explain Itself

A model that says “injury risk = 87%” is useless to a medical team if they can’t see **why**. So we used **SHAP** (SHapley Additive exPlanations) to crack open every prediction.

Where does the idea come from?

SHAP comes from game theory. Imagine a cooperative game where the “players” are the features (Age, Sleep, Training_Hours, ...). The “score” is the model’s prediction. Shapley values, invented by mathematician Lloyd Shapley in 1953, tell you exactly how much each player contributed to the team’s score, considering all possible orderings of who joined first. SHAP applies that same math to ML models.

What SHAP gives us

- **Global view** (summary plot): which features matter most across ALL players. In our project the leading features are Previous_Injury_Count, Fatigue_Index, Training_Hours_Per_Week and Recovery_Score.
- **Local view** (waterfall plot): for one specific player, exactly how each feature pushed the prediction up (toward injury) or down (toward healthy). Medical staff can show a player: *“Your previous injuries and high fatigue index added 0.4 to your risk; your good sleep subtracted 0.1.”*

Which features contribute the most, and why

- **Previous_Injury_Count** — old injuries predict new ones. Scar tissue, weakened ligaments, altered movement patterns.
- **Fatigue_Index** (engineered) — captures the dangerous combination of high load + low recovery. SHAP repeatedly highlights it.
- **Training_Hours_Per_Week** — overtraining is a known cause.
- **Recovery_Score** (engineered) — proxies the body’s ability to repair.
- **Sleep_Hours_Per_Night, Stress_Level_Score, Nutrition_Quality_Score** — lifestyle factors that move the needle.
- **Knee_Strength_Score, Hamstring_Flexibility, Balance_Test_Score** — when they are low, biomechanical risk goes up.

Why this matters in real life

A coach cannot defend a decision like “rest player 7 this weekend” with “the model said so”. With SHAP they can say: *“Player 7’s fatigue index is in the red zone, his last week training load is 1.6× his usual, and his previous injury count is 4. The model says 81% injury risk — these three factors account for 70% of that.”* That is auditable, defensible, regulator-friendly AI.

Chapter 10 — Machine Unlearning: How to Forget a Player

Now the cool part. A player files a deletion request. Their row must disappear from the model's "memory". We use a technique called **SISA**:

S.I.S.A. stands for:

- **S — Sharded:** split the training data into N chunks (we used 5 shards of 128 rows each).
- **I — Isolated:** train an independent model on each shard. No cross-talk.
- **S — Sliced:** each shard can be sub-sliced for further granularity (we kept this simple).
- **A — Aggregated:** at prediction time, average the probabilities from all 5 shard models. Vote like a small parliament.

Why does this make unlearning fast?

Because a single player's data lives in **exactly one shard**. When that player asks to be deleted, we only need to retrain that one shard model — not the entire ensemble. With 5 shards, a deletion typically costs ~1/5 of a full retrain. With 50 shards, ~1/50. The other shard models are untouched and still valid.

Our actual numbers

Method	Accuracy	ROC-AUC	Time (s)
Full retraining (baseline)	0.9563	0.9922	0.105
SISA unlearning	0.9375	0.9855	0.085

In our small dataset the time saving is modest (~19%). But the magic is in **how it scales**: on a real club dataset with hundreds of thousands of rows and a model that takes 1 hour to retrain, SISA with 50 shards turns a 1-hour retrain into a ~1-minute retrain. That is the headline.

And notice the accuracy/AUC barely drop. We honour the deletion request **and** keep the model just as good.

Where did the 54 "deletion requests" come from?

The synthetic *Deletion_Request* column we added: about 8% of players have it set to 1. In the training set that turned out to be 54 players. We removed all 54 of their rows and re-fit only the touched shards.

Chapter 11 — The Libraries (Tools of the Trade)

Every library used in the notebook is doing a specific job. Here is the shopping list:

Library	What it does for us
pandas	Reading the CSV and shaping the table — every df is a pandas DataFrame.
numpy	Underlying numerical engine — arrays, math, random seeds.
matplotlib	The base plotting library — every chart sits on top of it.
seaborn	Pretty statistical plots (boxplots, violinplots, heatmaps, KDEs).
scikit-learn	Train/test split, StandardScaler, Logistic Regression, Random Forest, all the metrics (accuracy,
xgboost	The XGBoost gradient-boosting model.
lightgbm	The LightGBM gradient-boosting model — used both as a classifier AND as the per-shard mod
shap	Computes Shapley values for tree models so we can explain predictions.
warnings, time, pathlib	Standard library helpers — silence noise, time the training, build cross-platform paths.

Chapter 12 — Tying It All Together

Let's walk through the whole pipeline as one continuous story:

- **1.** Load 800 footballers from a Kaggle CSV (plus our 3 synthetic ID columns).
- **2.** Check for missing values and duplicates — none found.
- **3.** One-hot encode the playing position into 4 binary columns.
- **4.** Hand-craft 4 sports-science features: ACWR, Recovery_Score, Fatigue_Index, Injury_Frequency_Score.
- **5.** Look at the data: boxplots, violin plots, KDEs, correlation heatmap.
- **6.** Stratified 80/20 split → 640 train, 160 test. Scale with StandardScaler.
- **7.** Train four models: Logistic Regression, Random Forest, XGBoost, LightGBM.
- **8.** Evaluate with Accuracy, Precision, Recall, F1, ROC-AUC. All models clear 0.92 accuracy.
- **9.** Use SHAP on LightGBM to explain global and per-player drivers.
- **10.** Build SISA: split training data into 5 shards, train one LightGBM per shard.
- **11.** Honour deletion requests (54 players): retrain only the affected shards, in 0.085 s vs 0.105 s for full retraining, with nearly identical AUC.
- **12.** Conclude: accurate injury prediction *and* privacy-preserving model maintenance, hand in hand.

Why this project is interesting beyond the grade

- **Sports clubs** get a usable injury early-warning system with auditable explanations.
- **Regulators / lawyers** get a working pattern for honouring GDPR-style deletion rights in ML pipelines — a problem the AI industry is still wrestling with.
- **Future researchers** can scale our SISA prototype from 5 shards to hundreds, on bigger datasets, with negligible accuracy loss.

Final scoreboard

- Best ROC-AUC: **Logistic Regression = 0.996**
- Best Accuracy: **Random Forest = 0.956**
- Best fit for downstream SHAP + SISA: **LightGBM**
- SISA time saved on deletion: **~19% on our small data**, scales to **~95%+ on real club-size data**
- SISA accuracy/AUC after unlearning: **0.94 / 0.986** — almost identical to baseline

In one sentence

We built a model that predicts football injuries with ~95% accuracy, explains every prediction with SHAP, and can forget any player in a fraction of the time a full retraining would take — all while keeping the model just as accurate.

Chapter 13 — 15 In-Depth Questions and Answers

These are the kinds of questions a curious reader, a teacher, or a viva examiner is likely to ask after going through this project. Each answer is kept simple but covers the real reasoning.

Q1. Why is predicting injuries treated as a classification problem and not a regression problem?

Because the question we are actually answering is a yes/no question: *will this player get injured next season?* The target column *Injury_Next_Season* only takes two values, 0 and 1. Regression would predict a continuous number (like "injury severity index = 2.7"), but our dataset does not contain such a label. We could later turn it into regression if we had injury *severity* data, but for now classification fits the available label perfectly.

Q2. The Kaggle dataset did not have Player_ID, Team_ID or Deletion_Request. Isn't adding synthetic columns cheating?

No, because we are not adding synthetic **labels** — the injury labels stay 100% real. We only added identity and a privacy flag, which any real club database would already have. The Kaggle dump was anonymised, so we re-attached fake IDs purely so we can simulate "Player X requests deletion". These columns are explicitly dropped from the feature matrix, so they cannot leak into the model and inflate scores.

Q3. Why did you choose a 50/50 balanced target? Real-world injuries are much rarer.

We used the dataset as Kaggle provided it. A balanced target is convenient for a course project because accuracy and ROC-AUC become meaningful immediately and we do not need SMOTE/under-sampling tricks. In production on a real club, the injury rate would be ~10–20%, and we would have to address class imbalance (class weights, focal loss, threshold tuning). The pipeline we built supports those changes; only the metric interpretation would shift (we would lean more on recall and PR-AUC).

Q4. Why did you use StandardScaler? Tree models don't need scaling.

Correct — Random Forest, XGBoost and LightGBM are scale-invariant. We scaled anyway for two practical reasons: (a) Logistic Regression in the same pipeline is very sensitive to scale; if one column ranged 0–300 (Reaction_Time_ms) and another ranged 0–1 (Warmup_Routine_Adherence), the larger one would dominate the loss; (b) keeping a single, uniformly scaled X matrix for every model makes the comparison fair and the code clean. Scaling tree models hurts nothing.

Q5. How exactly does the ACWR feature help predict injuries?

ACWR (Acute-Chronic Workload Ratio) captures **sudden spikes in training load**, which sports-science literature has linked to soft-tissue injuries. If a player normally trains 8 hours/week and suddenly trains 14, their tendons and muscles have not adapted. The ratio > 1.3 is a known warning zone. We computed it as *current week ÷ rolling 5-row mean*. It gives the model a single

number that encodes “unusual workload spike”, which raw training hours cannot do on their own.

Q6. Why didn’t you do hyperparameter tuning? Your boosting models could probably score higher.

Three reasons. (1) On this small, balanced dataset, default hyperparameters already clear ~95% accuracy and ~0.99 AUC — the headroom is tiny. (2) Tuning would have made the comparison between the four model families less fair, because then we would be benchmarking *tuning effort* instead of algorithm behaviour. (3) Overfitting risk: with only 640 training rows, aggressive tuning could end up memorising the validation fold. In a production rollout we would add GridSearchCV or Optuna with proper nested cross-validation.

Q7. Logistic Regression got the highest ROC-AUC. Doesn’t that mean the data is just linear and the boosting models are pointless here?

Almost, but not quite. The data *is* close to linearly separable thanks to the engineered features (especially Fatigue_Index and Injury_Frequency_Score), which already encode non-linear interactions. So Logistic Regression has very little left to learn beyond a straight line. The boosting models still earn their place: LightGBM is required for SHAP TreeExplainer (fast, exact attributions) and for SISA shard models. So we use boosting not because it wins AUC here but because it unlocks the explainability and unlearning capabilities downstream.

Q8. What is the difference between SHAP global summary plots and SHAP waterfall plots?

The **summary plot** is a bird’s-eye view: it shows, across all 160 test players, which features push predictions up or down the most, and whether high or low values of those features push toward injury. It answers: *“which features matter overall?”* The **waterfall plot** is a microscope on one single player: starting from the model’s baseline probability, it shows step by step how each feature added or subtracted from that probability until we reached the final prediction. It answers: *“why is THIS specific player at 81% risk?”*

Q9. What does TreeExplainer actually do under the hood?

TreeExplainer is a fast, exact algorithm for computing SHAP values specifically on tree-based models (Random Forest, XGBoost, LightGBM, CatBoost). For each prediction, it walks every tree in the ensemble and uses the path of decisions to fairly distribute credit to each feature. Without TreeExplainer, computing true Shapley values is exponential in the number of features — TreeExplainer reduces it to polynomial time using the tree structure, which is why we can run it on 160 test rows in seconds.

Q10. Why exactly 5 shards in SISA? Could you use more or fewer?

Five was a compromise. **More shards** → each shard has fewer rows, individual shard models become weaker (you can see that in our notebook: each shard scores ~0.88–0.91 alone), but unlearning is faster because a deleted row affects only one tiny shard. **Fewer shards** → each shard model is stronger, but the cost of retraining one shard is closer to a full retrain. Five shards on 640 training rows gives ~128 rows per shard, enough for a useful LightGBM, while still cutting deletion cost ~5×. On a production-scale dataset (say 50 000 rows), 50 shards of ~1000 rows would be a much better setting.

Q11. Your SISA only saved ~19% on retraining time. Is that really a win?

On 640 rows it is a small win — the full retrain itself is already a tenth of a second. The point of the experiment is to **prove the architecture works correctly** while losing very little accuracy (post-unlearning AUC 0.986 vs baseline 0.992). The real payoff is sub-linear scaling: on a 1-hour-to-train model, SISA with 50 shards turns each deletion into a ~1-minute retrain instead of an hour. The relative saving grows with dataset size and model complexity.

Q12. After SISA unlearning, can you mathematically prove the deleted player has no influence on the model?

Yes — that is the strongest selling point of SISA over approximate unlearning. Because each shard is **isolated** (the deleted player only appeared in one shard) and we **retrain that shard from scratch** on the remaining rows, the deleted player's data is provably gone from that shard model. The other shard models never saw the player to begin with. The aggregated ensemble therefore has the same statistical guarantee as a full retrain. Approximate unlearning methods (gradient-based, influence functions) only give you a *likely* removal, not a certified one.

Q13. What stops a club from just deleting the row from the CSV and calling it a day?

Deleting the row deletes the **data**, but not the **model** that was already trained on it. The model still carries the player's influence inside its learned parameters — it can re-derive features, embed memorised patterns, and expose them through predictions. GDPR-style frameworks are increasingly treating the trained model as derived personal data, so the deletion right must propagate to the model itself. SISA is one of the few practical techniques that lets you honour that obligation without burning a whole training cycle.

Q14. Why are precision and recall both important here, instead of just optimising one of them?

Both kinds of mistakes are costly in this domain. **Low precision** means many false alarms: healthy players get rested and matches are weakened for no reason — financial and competitive loss. **Low recall** means missed injuries: a player gets hurt that we could have warned about — career and welfare loss. F1 balances them with equal weight, and ROC-AUC summarises performance across all possible decision thresholds, which is why we chose ROC-AUC as the primary metric. If we were optimising for a specific medical-staff workflow, we could tilt the threshold toward whichever side they care about more.

Q15. If you had more time and data, what would you improve next?

Five concrete next steps: (1) Move from a static seasonal label to a **time-series** setup that predicts injury within the next 7/14/30 days using wearable data. (2) Replace synthetic Deletion_Request flags with a real consent-management feed. (3) Tune hyperparameters with Optuna and use 5-fold cross-validation. (4) Bump shards from 5 to 50–100 and add **slicing** (the second S in SISA) so each shard supports cheap incremental retraining within itself. (5) Add a fairness audit — check whether the model under-predicts injuries for specific positions or age groups, so that medical attention is distributed fairly.

Appendix — Glossary of Words We Used

Term	Plain meaning
Supervised learning	Training a model on examples where the correct answer is known.
Classification	Predicting a category (here: injured / not injured).
Target	The column we want to predict (Injury_Next_Season).
Feature	An input column the model uses to make a prediction.
Feature engineering	Creating smarter features from raw ones (e.g. Fatigue_Index).
One-hot encoding	Turning a categorical column (like Position) into 0/1 columns.
StandardScaler	Rescales numbers to mean 0, std 1.
Stratified split	Train/test split that preserves the class ratio in both halves.
Logistic Regression	Linear model that outputs a probability.
Random Forest	Ensemble of many decision trees that vote.
Gradient boosting	Sequential trees where each fixes the previous one's mistakes.
XGBoost / LightGBM	Two popular, fast implementations of gradient boosting.
Accuracy	Fraction of correct predictions.
Precision	Of predicted injuries, how many were real.
Recall	Of real injuries, how many we caught.
F1	Harmonic mean of precision and recall.
ROC-AUC	How well the model ranks positives above negatives across thresholds.
Confusion matrix	A 2x2 table of true vs predicted classes.
Overfitting	Memorising the training data instead of learning the pattern.
SHAP	A method to attribute a prediction to each feature using game theory.
Shapley value	The fair share of credit each feature deserves.
Machine unlearning	Removing a specific training example's influence from a model without full retraining.
SISA	Sharded, Isolated, Sliced, Aggregated — a practical unlearning architecture.
GDPR	European data-protection law that includes the 'right to be forgotten'.
ACWR	Acute-Chronic Workload Ratio — current load over recent average; >1.3 = risk.

End of document.

Built from *forgettable_athlete.ipynb* and *Forgettable_Athlete_Presentation.pptx* — Tokyo International University, Data Science & AI Project (Spring).